

SAT SOLUTECH DATASHARE

Uganda and Malta data isolation

Same eight Experience API endpoints.

Two regions. Two databases.

Two authentication options for client review.

Approach A — API Manager Client ID Enforcement

Approach B — JWT (JSON Web Token) validation

Prepared for client discussion

Experience layer (Exp) decides the database. Callers never send it.

1. What we are solving

SAT Datashare already exposes eight GET resources through one Experience API: Invoice, Order, Product, Route, SalesRep, Customer, Stock and Warehouse. Uganda and Malta will use **those same paths**. The only difference is which MySQL schema the System API queries (for example `sat_uganda` vs `sat_malta`).

Today the Experience API injects a single schema and supplier on every call. Clients do not send database. That is correct and must stay that way. If the caller were allowed to choose the schema, a Uganda integration could request Malta data.

The requirement

- Uganda must not read Malta rows.
- Malta must not read Uganda rows.
- Identify **who is calling** from the request itself.
- Set database (and supplier if needed) **inside the Experience API**.
- Do not duplicate the eight endpoints or the RAML resources.

Non-negotiable rule. The public client never supplies database or supplier. Those values are derived from authenticated identity in Exp, then passed privately to Process API and System API.

What we will not ask the client to send

Parameter	Who sends it	Why
fromDate, toDate, pageNumber	Uganda or Malta client	Business filters. Same for both regions.
database / supplier	Experience API only	Choosing schema is an authorisation decision, not an input.
region=uganda in query or header	Must not be trusted	Anyone can type the other region's name.

A query parameter or custom header such as X-Region: malta is not authentication. It is a label the caller controls.

2. Where identity is enforced

Traffic stays Exp → Process API (PAPI) → System API (SAPI) → MySQL. Only Exp is public. PAPI and SAPI must remain private (VPC, DLB or IP allowlist). If someone can call SAPI with `database=sat_malta`, Exp never gets a chance to stop them.

Layer	Role in this design
Experience API (Exp)	Authenticate the caller. Map identity → schema. Inject database and supplier. Same eight URLs.
Process API (PAPI)	Field mapping and paging only. Uses the schema Exp already chose. Not a public login point.
System API (SAPI)	Runs SQL as <code>schema.table</code> . Already supports a database query parameter from trusted callers.
MySQL	Two schemas on the same instance (simplest), or two hosts if operations require a hard split later.

Both options below keep this layering. They differ only in **how Exp recognises Uganda vs Malta**.

3. Approach A — Client ID Enforcement

Each region is registered as its own API Manager application. Uganda receives a `client_id` and `client_secret`. Malta receives a different pair. Anypoint API Manager applies the **Client ID Enforcement**

policy on the Experience API. A request without a valid pair is rejected before the Mule flow runs.

How a call works

- SAT Uganda stores only the Uganda client_id and secret in its integration.
- Every GET to /api/v1/Invoice (and the other seven paths) includes those credentials (typically headers client_id and client_secret).
- API Manager checks the pair against the contracts on this API. Invalid → HTTP 401.
- Exp reads the authenticated client_id, looks it up in configuration, and sets database = sat_uganda (and the Uganda supplier if it differs).
- Exp calls PAPI with that schema. PAPI calls SAPI. SQL runs only on Uganda tables.
- The same path with Malta credentials resolves to sat_malta only.

Configuration sketch (Experience API properties)

```
region.uganda.clientId = <<API Manager client_id for Uganda>>
region.uganda.database = sat_uganda
region.uganda.supplier = <<Uganda supplier name>>
```

```
region.malta.clientId = <<API Manager client_id for Malta>>
region.malta.database = sat_malta
region.malta.supplier = <<Malta supplier name>>
```

Unknown client_id after a successful policy check is treated as forbidden (HTTP 403). There is no default schema. There is no “if missing, use nobleoutlook”.

What the client application does

Item	Uganda	Malta
Base URL	Same Exp host	Same Exp host
Paths	Same eight GETs	Same eight GETs
Dates / paging	fromDate, toDate, pageNumber	fromDate, toDate, pageNumber
Credentials	Uganda client_id + secret	Malta client_id + secret
database query param	Not sent	Not sent

Strengths

- Fastest to deliver on CloudHub: one policy, a small lookup in Exp, no token server.
- Fits the current design: Exp already injects database; we only make that lookup identity-based.
- Operators can revoke one region in API Manager without redeploying SQL.
- No new login screens. Machine-to-machine integrations already understand client_id / secret.

Limits (must be stated to the client)

The client_id and secret are the identity. If Uganda obtains Malta's pair, the platform correctly treats them as Malta and returns Malta data. This is the same property as a stolen password. Client ID Enforcement stops “guess the other schema on the URL”. It does not stop a leaked secret.

Hardening if leakage is a realistic threat: store secrets only in each region's SAT runtime; rotate on suspected leak; add an IP allowlist on the policy so Uganda's credentials only work from Uganda egress IPs; or deploy two Experience applications so each runtime has only one schema in memory.

Operational extras

- SLA tiers in API Manager can throttle Uganda and Malta independently.
- Analytics show which contract called which resource.
- Secret rotation is an API Manager action plus an update in that region's SAT config — not a RAML change.

4. Approach B — JWT validation

A JWT is a short-lived, **signed** statement of identity. An Identity Provider (Anypoint, Okta, Azure AD, or similar) issues the token. Exp does not trust a region name typed by the caller. It trusts a region (or tenant) claim that only the issuer can sign.

How a call works

- Uganda's SAT application authenticates to the token endpoint (client credentials, or a user login if SAT later wants person-level tokens).
- The issuer returns a JWT. Typical claims:


```
{
  "sub": "sat-uganda",
  "region": "uganda",
  "aud": "exp-sat-datashare-api",
  "exp": 1778000000
}
```
- Uganda calls the same eight paths with `Authorization: Bearer <jwt>`.
- API Manager **JWT Validation** policy on Exp checks signature, expiry, issuer and audience. Invalid → HTTP 401.
- Exp reads region from the validated token and maps `uganda` → `sat_uganda`, `malta` → `sat_malta`.
- Exp injects database and supplier on the private call to PAPI. Downstream SQL never sees the other schema.

Why a caller cannot forge the other region

A JWT is three Base64 parts: header, payload, signature. Changing `"region": "malta"` to `"uganda"` breaks the signature. Exp rejects it. That is the difference from a query parameter.

What the client application does

Step	Uganda	Malta
Get token	Token URL + Uganda credentials (or user login)	Token URL + Malta credentials (or user login)
Call Exp	Bearer token on the same eight GETs	Bearer token on the same eight GETs
Token lifetime	Minutes (recommended 5–15)	Minutes (recommended 5–15)
database query param	Not sent	Not sent
Refresh	Request a new JWT when exp is near	Same

Two flavours of JWT (important for the client)

Flavour	Who the token represents	If credentials leak
Client-credentials JWT (machine)	The Uganda or Malta integration app	Attacker can mint new tokens for that region until the client secret is revoked. Still cannot mint the other region's tokens.
User JWT (authorisation code / SSO)	A named person in that organisation	Leak is one user session. Revoke that user. No shared country-wide password.

If the commercial concern is “the two countries might share secrets”, user JWTs are the stronger JWT variant. Client-credentials JWT is still strictly better than a static client_id on every request, because tokens expire.

Strengths

- Short-lived: a copied Bearer token dies when exp is reached.
- Region is inside a signed claim. Exp does not maintain a large client_id table if the IdP already emits region.
- Same pattern scales to more countries later (Kenya, etc.) without new URL paths.
- Industry-standard header; many SAT / partner stacks already send Bearer tokens.
- Can add scopes later (for example read:invoice) without changing the eight resource names.

Limits

- Needs a token issuer (Anypoint OpenID, Okta, Azure AD, or similar) and a JWT Validation policy. More moving parts than Client ID Enforcement.
- Clients must implement “get token, then call API, then refresh”. Slightly more work than sending two headers.
- A stolen **valid** JWT works until it expires. Keep TTL short; consider token revocation if the IdP supports it.
- A stolen **client secret** used at the token URL can still issue new JWTs for that region. JWT does not magically ignore leaked machine secrets — it contains the blast radius to that region and to a short window.

Experience API mapping after a valid JWT

```
region claim "uganda" → database sat_uganda, supplier <Uganda>
region claim "malta" → database sat_malta, supplier <Malta>
missing / unknown region → HTTP 403, no query executed
```

5. Side-by-side comparison

Topic	A — Client ID Enforcement	B — JWT
Identity on each request	client_id + client_secret (long-lived until rotated)	Bearer token (minutes)
Who decides region	Exp maps client_id → schema	IdP signs region/tenant claim; Exp maps claim → schema
Same eight endpoints	Yes	Yes
Caller sends database	No	No
Forge the other region without its secret	No	No (signature would fail)
Stolen other region's secret	Full access to that region until rotation	Can mint tokens for that region until secret is revoked; captured JWT dies at exp
Time to implement on this stack	Shortest	Short, but needs token URL + policy + client token handling
Extra platform pieces	API Manager applications + Client ID policy	Token issuer + JWT Validation policy + clock skew / JWKS
Revoke one country	Remove or reset that contract in API Manager	Revoke client or users at the IdP; tokens age out
Fits current Exp inject-database pattern	Yes — replace one property with a lookup	Yes — lookup keyed by claim instead of client_id
Partner familiarity	Very common on Mule APIs	Very common for Azure / Okta / OpenID shops
Future user-level audit	Application-level only	Natural if you issue user JWTs (sub = person)

Topic	A — Client ID Enforcement	B — JWT
Rate limit per country	SLA tier per client application	SLA per client, or per claim if policy supports it

6. Recommendation

Both approaches satisfy the functional requirement: same URLs, Exp chooses the schema, Uganda cannot select Malta's database as a parameter.

If the client's priority is...	Choose
Go live quickly, two machine integrations, secrets stay in each country's SAT app	Approach A — Client ID Enforcement
Tokens that expire, signed region claim, possible SSO / named users later, or SAT already uses OAuth	Approach B — JWT
Even a stolen secret must not work from the other country	Either approach plus IP allowlist, mTLS, or two separate Exp applications

Practical path. Start with **JWT client-credentials** if the client already has (or wants) a token URL. Start with **Client ID Enforcement** if they want the smallest change to SAT. Do not start by letting the caller pass region or database. Keep PAPI and SAPI off the public internet in both cases.

Suggested decision

For a two-country machine integration with Anypoint already in place, **Client ID Enforcement is the easiest** and is enough if secrets are not shared. **JWT is the better long-term standard** if the client expects more countries, short-lived credentials, or future user login. The Experience API mapping is almost identical; swapping A for B later does not require new Invoice/Order/... resources.

7. What does not change in the APIs

- The eight GET resources and their payloads / mappings.
- Paging (pageNumber, next).
- Date filters on Exp (fromDate, toDate).
- SAPI SQL shape: still schema.table with the schema name supplied by Exp.
- PAPI field mapping (STOREID, ProductID, and the rest).
- Exchange RAML resource names. Auth is a policy plus Exp configuration, not eight new paths.

Work on Exp is: attach the chosen API Manager policy, add the region → database map, refuse unknown identities, and keep ignoring any client-supplied database parameter.

8. Security checklist (both approaches)

Control	Why it matters
No public PAPI / SAPI	Stops bypassing Exp with database=theOtherCountry.
No database / supplier / region on the public query string	Caller must not choose the schema.
TLS on Exp	Secrets and tokens are not sent in clear text.
Secrets or private keys only in SAT + API Manager / IdP	Not in Postman collections shared across countries.

Control	Why it matters
Rotate on suspected leak	Client ID reset or IdP client secret rotation.
Short JWT TTL if Approach B	Limits the window for a copied Authorization header.
Logs: client_id or token sub, never the secret or full token	Audit who hit which resource without leaking credentials.
Optional IP allowlist / mTLS	Stolen credential used from the wrong network fails.

9. Implementation outline (after the client chooses)

Approach A

- Create two API Manager applications (Uganda, Malta) and contract them to the Experience API.
- Apply Client ID Enforcement on Exp (headers client_id / client_secret, or the equivalent for the policy version).
- Add region properties in Exp. Lookup by client_id. 403 if no match.
- Keep process-query-params as the single place that sets database and supplier — sourced from the lookup, not from a static property.
- Smoke-test: Uganda credentials return Uganda schema only; Malta the reverse; swapped credentials do not switch schema; missing credentials 401.

Approach B

- Stand up or reuse a token issuer. Register Uganda and Malta clients. Put region (or tenant) in the access token.
- Apply JWT Validation on Exp (JWKS or signed cert, issuer, audience, clock skew).
- Exp maps validated region → database / supplier. 403 if claim missing.
- Document token URL, required claims, and example Authorization header for each country.
- Same smoke tests as A, plus expired token → 401 and tampered region claim → 401.

10. Points to confirm with the client

- Are Uganda and Malta two **schemas on one MySQL** (simplest; SAPI already prefixes database.table) or two database servers (needs a second JDBC config in SAPI)?
- Is supplier the same string in both countries or different?
- Who holds the credentials today — one SAT team or two local teams?
- Is there already an IdP (Okta / Azure AD / Anypoint) we should use for JWT?
- Is IP allowlist acceptable (known SAT egress IPs per country)?
- Do they need named-user audit, or is application-level identity enough?

Bottom line for the meeting. The eight endpoints stay. The database is not a client input. Client ID Enforcement is the fastest Mule-native lock. JWT is the same lock with a signed, expiring card and a cleaner path to more countries and user-level tokens. Neither replaces the need to keep PAPI/SAPI private and to treat secrets as region-specific.